bias_trainer.py — technical

src/moe_pruning/training/bias_trainer.py · 352 lines · moe_pruning.training.bias_trainer

Purpose

Reference capture, layer-MSE training and the top-1 closed-form reference.

Formulas

```
L = (1/N) * sum over real tokens of ||F_orig(H) - F_repl(H)||_2^2 (sum over the hidden dimension, mean over tokens)
top-1 closed form: E = y_ref / g for routed tokens; b*_e = sum g^2 E / sum g^2
```

Step by step

Classes and functions in source order. Signatures, line ranges, raised exceptions and settings are extracted from the code; numbered steps describe what the code does.

`class LayerReference`

lines 49-58 · dataclass

Cached original inputs/outputs of one layer (real tokens of all examples).

Field	Type	Default
`layer_id`	str	
`example_ids`	list[int]	
`offsets`	list[int]	
`inputs`	torch.Tensor	
`outputs`	torch.Tensor	

`LayerReference.sequence`

```
def sequence(self, i: int) -> tuple[torch.Tensor, torch.Tensor]
```

lines 56-58

Inputs and outputs of example i.

`save_layer_reference`

```
def save_layer_reference(ref: LayerReference, cache_dir: Path, meta: dict) -> None
```

lines 65-76

Write a layer's reference cache and its metadata.

`load_layer_reference`

```
def load_layer_reference(cache_dir: Path, layer_id: str) -> tuple[LayerReference, dict]
```

lines 79-83

Read a layer's reference cache and its metadata.

`cache_is_valid`

```
def cache_is_valid(cache_dir: Path, layer_ids: list[str], expected_meta: dict) -> bool
```

lines 86-93

True if cached references exist for all layers with matching metadata.

`capture_layer_references`

```
def capture_layer_references(model: nn.Module, adapter: MoEAdapter, layers: list[MoELayerInfo], batches, device: torch.device) -> dict[str, LayerReference]
```

lines 97-135 · raises RuntimeError

Run the original model and keep (input, output) of each layer for real tokens.

1. Instrument the selected layers of the ORIGINAL model; for each batch row take the first n real tokens (right padding guaranteed; checked) of MoE input and output, store in model dtype if 16-bit else float32, on CPU.

2. Concatenate per layer into `inputs [T, D]`, `outputs [T, D]` with per-example offsets.

`masked_sequence_batch`

```
def masked_sequence_batch(ref: LayerReference, rows: list[int], device: torch.device, dtype:
torch.dtype) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]
```

lines 141-156

Right-pad sequences with zeros. Returns x [B,S,D], y [B,S,D], mask [B,S].

1. Right-pad the chosen sequences with zeros into `x [B, S, D]` (model dtype) and `y` (float32) plus a mask; move to device.

`reconstruction_loss`

```
def reconstruction_loss(y_ref: torch.Tensor, y_hat: torch.Tensor, mask: torch.Tensor) ->
tuple[torch.Tensor, int]
```

lines 159-163

(1/N) * sum over real tokens of ||y_ref - y_hat||_2^2, and N.

1. Squared error summed over D, masked, divided by the number of real tokens.

`evaluate_reconstruction`

```
def evaluate_reconstruction(model: nn.Module, adapter: MoEAdapter, layer: MoELayerInfo, ref:
LayerReference, rows: list[int], device: torch.device, batch_size: int) -> float
```

lines 167-178

Layer MSE of the current biases on a set of examples (no gradients).

`closed_form_biases`

```
def closed_form_biases(model: nn.Module, adapter: MoEAdapter, layer: MoELayerInfo, ref: LayerReference,
rows: list[int], expert_ids: list[int], device: torch.device, batch_size: int) -> dict[int, torch.Tensor
| None]
```

lines 182-207 · raises NotImplementedError

Minimiser of the layer MSE on `rows`: b*_e = sum g^2 E / sum g^2.

1. Top-1 only. Route the cached inputs with the (unchanged) router; for tokens routed to each removed expert recover E = y/g and accumulate g^2 E and g^2 in float64 on CPU; divide.

`class LayerTrainingResult`

lines 214-232 · dataclass

Losses, closed-form reference, bias changes and history of one layer.

Field	Type	Default
`layer_id`	str	
`expert_ids`	list[int]	
`trainable_parameters`	int	
`steps`	int	
`skipped_steps_without_routed_tokens`	int	
`initial_train_loss`	float	
`initial_validation_loss`	float	
`final_train_loss`	float	
`final_validation_loss`	float	
`closed_form_train_loss`	float | None	
`closed_form_validation_loss`	float | None	
`bias_change_norm`	dict[str, float]	
`distance_to_closed_form`	dict[str, float | None]	
`history`	list[dict]	field(default_factory=list)

Field	Type	Default
seconds	float	0.0

LayerTrainingResult.to_dict

```
def to_dict(self) -> dict
```

lines 231-232

Convert to a plain dictionary for JSON reports.

train_layer_biases

```
def train_layer_biases(model: nn.Module, adapter: MoEAdapter, layer: MoELayerInfo, expert_ids:
list[int], ref: LayerReference, train_rows: list[int], val_rows: list[int], device: torch.device,
learning_rate: float, steps: int, batch_size: int, eval_interval: int, seed: int) -> LayerTrainingResult
```

lines 245-352 · raises RuntimeError

Optimise all BiasExpert biases of one layer jointly with Adam.

1. Device = the layer's own device; eval mode (no dropout); collect the layer's BiasExpert parameters (must be trainable).
2. Closed-form reference (top-1 only): temporarily set biases to b*, measure train/val loss, restore.
3. Initial train/val loss; Adam over the biases.
4. Each step: refill and shuffle a row queue with a seeded RNG when needed, take `BIAS_BATCH_SIZE` rows, forward only the MoE module, loss; if the loss does not depend on the biases (no token routed to a replaced expert) skip the step instead of letting Adam momentum move them; else backward and step.
5. Every `BIAS_EVAL_INTERVAL` steps and at the end: full train/val loss into the history.
6. Return losses, closed-form losses, bias change norms, distance to the closed form, history and timing.

Errors raised

Function	Exception	Message (f-string fields in braces)
capture_layer_references	RuntimeError	Expected right-padded sequences
closed_form_biases	NotImplementedError	closed-form reference implemented for top-1 routing
_layer_biases	TypeError	{layer.layer_id} expert {e} is not a BiasExpert
train_layer_biases	RuntimeError	Bias parameters must be trainable (call freeze_all_but_biases first)
on_record	RuntimeError	Expected right-padded sequences